In [1]:

```
import pandas as pd
import numpy as np
import os
```

In [2]:

```
file_path = r"C:\Users\2025a.LAPTOP\Downloads\finkraft\data\Finkraft_Travel_Expense_GST_
df = pd.read_excel(
    file_path,
    sheet_name="Raw_Expenses"
)

print("Dataset loaded successfully!")
```

```
Dataset loaded successfully!
```

In [3]:

```
df.head(5)
```

Out[3]:

	Claim_ID	Expense_Date	Employee	Department	Client	City	Expense_Category	Vendor	Inv
0	TRV-0001	2026-07-02	Aarav Mehta	Sales	Northstar Retail Pvt Ltd	Mumbai	Hotel	Harbor Stay Hotels	2(
1	TRV-0002	2026-07-03	Isha Shah	Consulting	BluePeak Finance Ltd	Pune	Cab	CityRide Mobility	2(
2	TRV-0003	2026-07-04	Rohan Patil	Operations	Aster Travel Services	Nashik	Train	IRCTC / Rail Vendor	RV
3	TRV-0004	2026-07-05	Neha Kulkarni	Sales	Northstar Retail Pvt Ltd	Mumbai	Meals	Metro Bistro	
4	TRV-0005	2026-07-06	kabir joshi	Consulting	BluePeak Finance Ltd	Thane	Cab	CityRide Mobility	2(

In [4]:

```
df.shape[0]
```

Out[4]:

```
32
```

In [5]:

```
df.shape[1]
```

Out[5]:

```
19
```

In [6]:

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 32 entries, 0 to 31
Data columns (total 19 columns):
 #   Column              Non-Null Count  Dtype
---  ------              --------------  -----
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
 0   Claim_ID            32 non-null     object
 1   Expense_Date        32 non-null     object
 2   Employee            31 non-null     object
 3   Department          32 non-null     object
 4   Client              31 non-null     object
 5   City                32 non-null     object
 6   Expense_Category    32 non-null     object
 7   Vendor              31 non-null     object
 8   Invoice_No          30 non-null     object
 9   Taxable_Amount_INR  31 non-null     object
 10  CGST_INR            32 non-null     float64
 11  SGST_INR            32 non-null     float64
 12  IGST_INR            31 non-null     float64
 13  GST_Total_INR       31 non-null     float64
 14  Invoice_Total_INR   31 non-null     object
 15  Claim_Status        32 non-null     object
 16  Business_Purpose    32 non-null     object
 17  Invoice_Check       0 non-null      float64
 18  GSTIN_Sample        32 non-null     object
dtypes: float64(5), object(14)
memory usage: 4.9+ KB
```

In [7]:

```
df.describe()
```

Out[7]:

	CGST_INR	SGST_INR	IGST_INR	GST_Total_INR	Invoice_Check
count	32.000000	32.000000	31.000000	31.000000	0.0
mean	126.859375	124.875000	201.251613	462.348387	NaN
std	192.141381	190.598209	382.521930	434.048784	NaN
min	0.000000	0.000000	0.000000	0.000000	NaN
25%	0.000000	0.000000	0.000000	112.600000	NaN
50%	0.000000	0.000000	0.000000	225.000000	NaN
75%	324.000000	303.750000	187.200000	846.000000	NaN
max	549.000000	549.000000	1278.000000	1278.000000	NaN

In [8]:

```
# Identifying the missing values
missing_values = df.isnull().sum()

missing_report = pd.DataFrame({
    "Column": missing_values.index,
    "Missing Count": missing_values.values,
    "Missing Percentage" : (missing_values.values / len(df) * 100).round(2)
})

missing_report = missing_report.sort_values(
    by="Missing Count",
    ascending=False
)

missing_report
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

Out[8]:

	Column	Missing Count	Missing Percentage
17	Invoice_Check	32	100.00
8	Invoice_No	2	6.25
9	Taxable_Amount_INR	1	3.12
12	IGST_INR	1	3.12
2	Employee	1	3.12
4	Client	1	3.12
7	Vendor	1	3.12
14	Invoice_Total_INR	1	3.12
13	GST_Total_INR	1	3.12
16	Business_Purpose	0	0.00
15	Claim_Status	0	0.00
0	Claim_ID	0	0.00
11	SGST_INR	0	0.00
10	CGST_INR	0	0.00
1	Expense_Date	0	0.00
6	Expense_Category	0	0.00
5	City	0	0.00
3	Department	0	0.00
18	GSTIN_Sample	0	0.00

In [9]:

```
# Finding duplicate rows
print(
    "Duplecated rows:",
    df.duplicated().sum()
)
```

```
Duplecated rows: 0
```

In [10]:

```
# Duplicate claim IDs
duplicate_claims = df[
    df.duplicated(subset=["Claim_ID"], keep=False)
    ]
duplicate_claims
```

Out[10]:

	Claim_ID	Expense_Date	Employee	Department	Client	City	Expense_Category	Vendor	Invoice_No

In [11]:

```
# Investigating repeated invoice numbers
duplicate_invoices = df[
    df["Invoice_No"].notna()
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
    & df["Invoice_No"].astype(str).str.strip().ne("")
    & df["Invoice_No"].duplicated(keep=False)
    ]

duplicate_invoices[
    [
        "Claim_ID",
        "Employee",
        "Vendor",
        "Invoice_No",
        "Expense_Date",
        "Invoice_Total_INR"
    ]
    ]
```

Out[11]:

	Claim_ID	Employee	Vendor	Invoice_No	Expense_Date	Invoice_Total_INR
9	TRV-0010	Kabir Joshi	IRCTC / Rail Vendor	RV-260711-16	2026-07-11	1450
16	TRV-0017	Isha Shah	Harbor Stay Hotels	HS-260718-42	2026-07-18	7198
26	TRV-0027	Isha Shah	Harbor Stay Hotels	HS-260718-42	2026-07-28	6490
30	TRV-0031	Kabir Joshi	IRCTC / Rail Vendor	RV-260711-16	2026-07-11	1450

In [12]:

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 32 entries, 0 to 31
Data columns (total 19 columns):
 #   Column              Non-Null Count  Dtype  
---  ------              --------------  -----  
 0   Claim_ID            32 non-null     object 
 1   Expense_Date        32 non-null     object 
 2   Employee            31 non-null     object 
 3   Department          32 non-null     object 
 4   Client              31 non-null     object 
 5   City                32 non-null     object 
 6   Expense_Category    32 non-null     object 
 7   Vendor              31 non-null     object 
 8   Invoice_No          30 non-null     object 
 9   Taxable_Amount_INR  31 non-null     object 
 10  CGST_INR            32 non-null     float64
 11  SGST_INR            32 non-null     float64
 12  IGST_INR            31 non-null     float64
 13  GST_Total_INR       31 non-null     float64
 14  Invoice_Total_INR   31 non-null     object 
 15  Claim_Status        32 non-null     object 
 16  Business_Purpose    32 non-null     object 
 17  Invoice_Check       0 non-null      float64
 18  GSTIN_Sample        32 non-null     object 
dtypes: float64(5), object(14)
memory usage: 4.9+ KB
```

In [13]:

```
# Finding Inconsistent text
text_columns = [
    "Employee",
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
    "City",
    "Expense_Category",
    "Claim_Status"
]

for column in text_columns:
    print(f"\nUnique values in {column}:")
    print(df[column].dropna().unique())
```

```
Unique values in Employee:
['Aarav Mehta' 'Isha Shah' 'Rohan Patil' 'Neha Kulkarni' '  kabir joshi '
 'Kabir Joshi' 'Riya Desai']

Unique values in City:
['Mumbai' 'Pune' 'Nashik' 'Thane' 'Delhi' 'mumbai ' 'Bengaluru']

Unique values in Expense_Category:
['Hotel' 'Cab' 'Train' 'Meals' 'Flight' ' meals ']

Unique values in Claim_Status:
['Approved' 'Pending' 'Rejected' 'approved ' 'In Process??']
```

In [14]:

```
df.head()
```

Out[14]:

	Claim_ID	Expense_Date	Employee	Department	Client	City	Expense_Category	Vendor	Inv
0	TRV-0001	2026-07-02	Aarav Mehta	Sales	Northstar Retail Pvt Ltd	Mumbai	Hotel	Harbor Stay Hotels	2(
1	TRV-0002	2026-07-03	Isha Shah	Consulting	BluePeak Finance Ltd	Pune	Cab	CityRide Mobility	2(
2	TRV-0003	2026-07-04	Rohan Patil	Operations	Aster Travel Services	Nashik	Train	IRCTC / Rail Vendor	RV
3	TRV-0004	2026-07-05	Neha Kulkarni	Sales	Northstar Retail Pvt Ltd	Mumbai	Meals	Metro Bistro	
4	TRV-0005	2026-07-06	kabir joshi	Consulting	BluePeak Finance Ltd	Thane	Cab	CityRide Mobility	2(

In [15]:

```
# Checking the date column
print(df["Expense_Date"].head(15))
print("Date column dtype:", df["Expense_Date"].dtype)
```

```
0     2026-07-02
1     2026-07-03
2     2026-07-04
3     2026-07-05
4     2026-07-06
5     2026-07-07
6     2026-07-08
7     2026-07-09
8     10/07/2026
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
9     2026-07-11
10    2026-07-12
11    2026-07-13
12    2026-07-14
13    2026-07-15
14    2026-07-16
Name: Expense_Date, dtype: object
Date column dtype: object
```

In [16]:

```
# Creating a working copy
clean_df = df.copy()

print("Raw rows:", len(df))
print("Working rows:", len(clean_df))
```

```
Raw rows: 32
Working rows: 32
```

In [17]:

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 32 entries, 0 to 31
Data columns (total 19 columns):
 #   Column              Non-Null Count  Dtype  
---  ------              --------------  -----  
 0   Claim_ID            32 non-null     object 
 1   Expense_Date        32 non-null     object 
 2   Employee            31 non-null     object 
 3   Department          32 non-null     object 
 4   Client              31 non-null     object 
 5   City                32 non-null     object 
 6   Expense_Category    32 non-null     object 
 7   Vendor              31 non-null     object 
 8   Invoice_No          30 non-null     object 
 9   Taxable_Amount_INR  31 non-null     object 
 10  CGST_INR            32 non-null     float64
 11  SGST_INR            32 non-null     float64
 12  IGST_INR            31 non-null     float64
 13  GST_Total_INR       31 non-null     float64
 14  Invoice_Total_INR   31 non-null     object 
 15  Claim_Status        32 non-null     object 
 16  Business_Purpose    32 non-null     object 
 17  Invoice_Check       0 non-null      float64
 18  GSTIN_Sample        32 non-null     object 
dtypes: float64(5), object(14)
memory usage: 4.9+ KB
```

In [18]:

```
# Standarizing text fields
text_columns = [
    "Employee",
    "Department",
    "Client",
    "City",
    "Expense_Category",
    "Vendor",
    "Claim_Status",
    "Invoice_No",
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
    "Business_Purpose"
]

for column in text_columns:
    clean_df[column] = (
        clean_df[column]
        .astype("string")
        .str.strip()
        .replace("",pd.NA)
    )
```

In [19]:

```
# Standarizing some selected columns
clean_df["Employee"] = clean_df["Employee"].str.title()

clean_df["City"] = clean_df["City"].str.title()

clean_df["Expense_Category"] = clean_df["Expense_Category"].str.title()

clean_df["Claim_Status"] = clean_df["Claim_Status"].str.title()
```

In [20]:

```
df.head()
```

Out[20]:

	Claim_ID	Expense_Date	Employee	Department	Client	City	Expense_Category	Vendor	Inv
0	TRV-0001	2026-07-02	Aarav Mehta	Sales	Northstar Retail Pvt Ltd	Mumbai	Hotel	Harbor Stay Hotels	2(
1	TRV-0002	2026-07-03	Isha Shah	Consulting	BluePeak Finance Ltd	Pune	Cab	CityRide Mobility	2(
2	TRV-0003	2026-07-04	Rohan Patil	Operations	Aster Travel Services	Nashik	Train	IRCTC / Rail Vendor	RV
3	TRV-0004	2026-07-05	Neha Kulkarni	Sales	Northstar Retail Pvt Ltd	Mumbai	Meals	Metro Bistro	
4	TRV-0005	2026-07-06	kabir joshi	Consulting	BluePeak Finance Ltd	Thane	Cab	CityRide Mobility	2(

In [21]:

```
# Cleaning and  validating dates
date_text = (
    clean_df["Expense_Date"]
    .astype("string")
    .str.strip()
)

parsed_dates = pd.to_datetime(
    date_text,
    errors="coerce",
    format="mixed",
    dayfirst=True
)
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
clean_df["Expense_Date_Original"] = date_text
clean_df["Expense_Date"] = parsed_dates
```

In [22]:

```
clean_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 32 entries, 0 to 31
Data columns (total 20 columns):
 #   Column                 Non-Null Count  Dtype
---  ------                 --------------  -----
 0   Claim_ID               32 non-null     object
 1   Expense_Date           31 non-null     datetime64[ns]
 2   Employee               31 non-null     string
 3   Department             32 non-null     string
 4   Client                 31 non-null     string
 5   City                   32 non-null     string
 6   Expense_Category       32 non-null     string
 7   Vendor                 31 non-null     string
 8   Invoice_No             30 non-null     string
 9   Taxable_Amount_INR     31 non-null     object
 10  CGST_INR               32 non-null     float64
 11  SGST_INR               32 non-null     float64
 12  IGST_INR               31 non-null     float64
 13  GST_Total_INR          31 non-null     float64
 14  Invoice_Total_INR      31 non-null     object
 15  Claim_Status           32 non-null     string
 16  Business_Purpose       32 non-null     string
 17  Invoice_Check          0 non-null      float64
 18  GSTIN_Sample           32 non-null     object
 19  Expense_Date_Original  32 non-null     string
dtypes: datetime64[ns](1), float64(5), object(4), string(10)
memory usage: 5.1+ KB
```

In [23]:

```
invalid_dates = clean_df[
    clean_df["Expense_Date"].isna()
    & date_text.notna()
    & date_text.ne("")
]

invalid_dates[
    ["Claim_ID", "Expense_Date_Original"]
]
```

Out[23]:

	Claim_ID	Expense_Date_Original
22	TRV-0023	not available

In [24]:

```
# Converting monetary columns to numeric values
def clean_numeric(series):
    cleaned = (
        series.astype("string")
        .str.strip()
        .str.replace("₹", "", regex=False)
        .str.replace(",", "", regex=False)
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
        .str.replace(r"\s+", "", regex=True)
    )

    cleaned = cleaned.replace(
    ["", "nan", "None", "<NA>"],
    pd.NA
    )

    return pd.to_numeric(
    cleaned,
    errors="coerce"
    )
```

In [25]:

```
numeric_columns = [
    "Taxable_Amount_INR",
    "CGST_INR",
    "SGST_INR",
    "IGST_INR",
    "GST_Total_INR",
    "Invoice_Total_INR"
]

for column in numeric_columns:
    clean_df[column] = clean_numeric(clean_df[column])
```

In [26]:

```
clean_df[numeric_columns].dtypes
```

Out[26]:

```
Taxable_Amount_INR      Int64
CGST_INR              Float64
SGST_INR              Float64
IGST_INR              Float64
GST_Total_INR         Float64
Invoice_Total_INR     Float64
dtype: object
```

In [27]:

```
# Finding records where numeric conversion failed
clean_df[
    clean_df["Taxable_Amount_INR"].isna()
][
    [
        "Claim_ID",
        "Taxable_Amount_INR"
    ]
]
```

Out[27]:

	Claim_ID	Taxable_Amount_INR
31	TRV-0032	<NA>

In [28]:

```
# Validating GST and invoice totals
clean_df["Calculated_GST_INR"] = (
    clean_df["CGST_INR"].fillna(0)
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
    + clean_df["SGST_INR"].fillna(0)
    + clean_df["IGST_INR"].fillna(0)
)
```

In [29]:

```
# Comparing calculated amt with recorder GST total
clean_df["GST_Check"] = np.where(
    clean_df["GST_Total_INR"].isna(),
    "Missing recorded GST total",
    np.where(
        np.isclose(
            clean_df["GST_Total_INR"],
            clean_df["Calculated_GST_INR"],
            atol=0.01
        ),
        "Pass",
        "Review"
    )
)
```

In [30]:

```
clean_df.head()
```

Out[30]:

	Claim_ID	Expense_Date	Employee	Department	Client	City	Expense_Category	Vendor	Inv
0	TRV-0001	2026-07-02	Aarav Mehta	Sales	Northstar Retail Pvt Ltd	Mumbai	Hotel	Harbor Stay Hotels	2(
1	TRV-0002	2026-07-03	Isha Shah	Consulting	BluePeak Finance Ltd	Pune	Cab	CityRide Mobility	2(
2	TRV-0003	2026-07-04	Rohan Patil	Operations	Aster Travel Services	Nashik	Train	IRCTC / Rail Vendor	RV
3	TRV-0004	2026-07-05	Neha Kulkarni	Sales	Northstar Retail Pvt Ltd	Mumbai	Meals	Metro Bistro	
4	TRV-0005	2026-07-06	Kabir Joshi	Consulting	BluePeak Finance Ltd	Thane	Cab	CityRide Mobility	2(

5 rows × 22 columns

In [31]:

```
clean_df.loc[
    clean_df["GST_Check"] != "Pass",
    [
        "Claim_ID",
        "CGST_INR",
        "SGST_INR",
        "IGST_INR",
        "GST_Total_INR",
        "Calculated_GST_INR",
        "GST_Check"
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
    ]
]
```

Out[31]:

	Claim_ID	CGST_INR	SGST_INR	IGST_INR	GST_Total_INR	Calculated_GST_INR	GST_Check
14	TRV-0015	50.0	37.5	0.0	75.0	87.5	Review
20	TRV-0021	351.0	300.0	0.0	702.0	651.0	Review
31	TRV-0032	0.0	0.0	<NA>	<NA>	0.0	Missing recorded GST total

In [32]:

```
# Validating Invoice totals
clean_df["Expected_Invoice_Total_INR"] =(
    clean_df["Taxable_Amount_INR"]
    + clean_df["Calculated_GST_INR"]
)
```

In [33]:

```
clean_df["Invoice_Total_Check"] = np.where(
    clean_df["Invoice_Total_INR"].isna(),
    "Missing invoice total",
    np.where(
        clean_df["Expected_Invoice_Total_INR"].isna(),
        "Cannot validate",
        np.where(
            np.isclose(
                clean_df["Invoice_Total_INR"],
                clean_df["Expected_Invoice_Total_INR"],
                atol=0.01
            ),
            "Pass",
            "Mismatch"
        )
    )
)
```

In [34]:

```
invoice_mismatches = clean_df[
    clean_df["Invoice_Total_Check"] != "Pass"
    ]

invoice_mismatches[
    [
        "Claim_ID",
        "Taxable_Amount_INR",
        "Invoice_Total_INR",
        "Calculated_GST_INR",
        "Expected_Invoice_Total_INR",
        "Invoice_Total_Check"
    ]
    ]
```

Out[34]:

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

	Claim_ID	Taxable_Amount_INR	Invoice_Total_INR	Calculated_GST_INR	Expected_Invoice_Total_INR
14	TRV-0015	750	825.0	87.5	837.5
18	TRV-0019	890	900.0	160.2	1050.2
20	TRV-0021	3900	4602.0	651.0	4551.0
27	TRV-0028	-720	849.6	129.6	-590.4
31	TRV-0032	<NA>	<NA>	0.0	<NA>

In [35]:

```
# Creating an exception log

# Intializing the log
exception = []
```

In [36]:

```
# Flagging missing critical fields
required_fields = [
    "Expense_Date",
    "Employee",
    "Client",
    "Vendor",
    "Invoice_No",
    "Taxable_Amount_INR"
]

for column in required_fields:
    missing_rows = clean_df[
        clean_df[column].isna()
        ]

    for _, row in missing_rows.iterrows():
        exception.append({
            "Claim_ID": row["Claim_ID"],
            "Issue_Type": "Missing Required Field",
            "Field": column,
            "Orginal_Value": None,
            "Suggested_Action": (
                "Verify against the source invoice"
                "or expense submission"
            ),
            "Review_Status": "Open"
        })
```

In [37]:

```
# Flagging invalid dates
for _, rows in clean_df[
    clean_df["Expense_Date"].isna()
    ].iterrows():

    exception.append({
        "Claim_ID": row["Claim_ID"],
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
        "Issue_Type": "Invalid or Missing Date",
        "Field": "Expense_Date",
        "Original_Value": row["Expense_Date_Original"],
        "Suggested_Action": "Verify source document",
        "Review_Status": "Open"
    })
```

In [38]:

```
# Flagging invoice mismatchea
for _, row in clean_df[
    clean_df["Invoice_Total_Check"] != "Pass"
    ].iterrows():

    exception.append({
        "Claim_ID": row["Claim_ID"],
        "Issue_Type": "Invoice Total Validation",
        "Field": "Invoice_Total_INR",
        "Original_Value": row["Invoice_Total_INR"],
        "Suggested_Action": (
            "Compare recorded total with taxable value"
            "and calculated GST"
        ),
        "Review_Status": "Open"
    })
```

In [39]:

```
# Flagging GST Inconsistency
for _, row in clean_df[
    clean_df["GST_Check"] != "Pass"
    ].iterrows():

    exception.append({
        "Claim_ID": row["Claim_ID"],
        "Issue_Type": "GST Validation",
        "Field": "GST_Total_INR",
        "Original_Value": row["GST_Total_INR"],
        "Suggested_Action": (
            "Verify CGST, SGST, IGST, and recorded GST Total"
        ),
        "Review_Status": "Open"
    })
```

In [42]:

```
exception_df = pd.DataFrame(exception)

exception_df.head(15)
```

Out[42]:

	Claim_ID	Issue_Type	Field	Orginal_Value	Suggested_Action	Review_Status	Origin
0	TRV-0023	Missing Required Field	Expense_Date	NaN	Verify against the source invoiceor expense su...	Open	
1	TRV-0013	Missing Required Field	Employee	NaN	Verify against the source invoiceor expense su...	Open	
2	TRV-0024	Missing Required	Client	NaN	Verify against the source invoiceor	Open	

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

	Claim_ID	Issue_Type	Field	Orginal_Value	Suggested_Action	Review_Status	Origin
		Field			expense su...		
3	TRV-0022	Missing Required Field	Vendor	NaN	Verify against the source invoiceor expense su...	Open	
4	TRV-0004	Missing Required Field	Invoice_No	NaN	Verify against the source invoiceor expense su...	Open	
5	TRV-0032	Missing Required Field	Invoice_No	NaN	Verify against the source invoiceor expense su...	Open	
6	TRV-0032	Missing Required Field	Taxable_Amount_INR	NaN	Verify against the source invoiceor expense su...	Open	
7	TRV-0032	Invalid or Missing Date	Expense_Date	NaN	Verify source document	Open	20
8	TRV-0015	Invoice Total Validation	Invoice_Total_INR	NaN	Compare recorded total with taxable valueand c...	Open	
9	TRV-0019	Invoice Total Validation	Invoice_Total_INR	NaN	Compare recorded total with taxable valueand c...	Open	
10	TRV-0021	Invoice Total Validation	Invoice_Total_INR	NaN	Compare recorded total with taxable valueand c...	Open	
11	TRV-0028	Invoice Total Validation	Invoice_Total_INR	NaN	Compare recorded total with taxable valueand c...	Open	
12	TRV-0032	Invoice Total Validation	Invoice_Total_INR	NaN	Compare recorded total with taxable valueand c...	Open	
13	TRV-0015	GST Validation	GST_Total_INR	NaN	Verify CGST, SGST, IGST, and recorded GST Total	Open	
14	TRV-0021	GST Validation	GST_Total_INR	NaN	Verify CGST, SGST, IGST, and recorded GST Total	Open	

In [43]:

```
dup
```

Out[43]:

	Claim_ID	Expense_Date	Employee	Department	Client	City	Expense_Category	Vendor
14	TRV-0015	2026-07-16	Kabir Joshi	Consulting	BluePeak Finance Ltd	Nashik	Meals	Metro Bistro
18	TRV-0019	2026-07-20	Neha Kulkarni	Sales	Cedar Logistics Ltd	Mumbai	Cab	CityRide Mobility

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

	Claim_ID	Expense_Date	Employee	Department	Client	City	Expense_Category	Vendor
20	TRV-0021	2026-07-22	Aarav Mehta	Sales	BluePeak Finance Ltd	Mumbai	Hotel	UrbanNest Suites
27	TRV-0028	2026-07-29	Rohan Patil	Operations	Cedar Logistics Ltd	Mumbai	Cab	CityRide Mobility
31	TRV-0032	2026-07-31	Riya Desai	Operations	Cedar Logistics Ltd	Pune	Cab	CityRide Mobility

5 rows × 24 columns

In [44]:

```
# Finding duplicate invoice numbers
duplicate_mask = (
    clean_df["Invoice_No"].notna()
    & clean_df["Invoice_No"].duplicated(keep=False)
)

duplicate_review = clean_df.loc[
    duplicate_mask,
    [
        "Claim_ID",
        "Employee",
        "Vendor",
        "Invoice_No",
        "Expense_Date",
        "Taxable_Amount_INR",
        "Invoice_Total_INR"
    ]
    ]

duplicate_review
```

Out[44]:

	Claim_ID	Employee	Vendor	Invoice_No	Expense_Date	Taxable_Amount_INR	Invoice_Total_INR
9	TRV-0010	Kabir Joshi	IRCTC / Rail Vendor	RV-260711-16	2026-07-11	1450	1450.0
16	TRV-0017	Isha Shah	Harbor Stay Hotels	HS-260718-42	2026-07-18	6100	7198.0
26	TRV-0027	Isha Shah	Harbor Stay Hotels	HS-260718-42	2026-07-28	5500	6490.0
30	TRV-0031	Kabir Joshi	IRCTC / Rail Vendor	RV-260711-16	2026-07-11	1450	1450.0

In [45]:

```
# Exporting clean working data
output_folder = r"C:\Users\2025a.LAPTOP\Downloads\finkraft\data"
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
os.makedirs(output_folder, exist_ok=True)

clean_df.to_csv(
    os.path.join(output_folder, "cleaned_expenses.csv"),
    index=False
)
```

In [46]:

```
# Exporting exception log
exception_df.to_csv(
    os.path.join(output_folder, "exception_log.csv"),
    index=False
)
```

In [47]:

```
report_path = os.path.join(
    output_folder,
    "data_quality_report.xlsx"
)

with pd.ExcelWriter(
    report_path,
    engine="openpyxl"
) as writer:

    clean_df.to_excel(
        writer,
        sheet_name="Exceptions_Log",
        index=False
    )

    duplicate_review.to_excel(
        writer,
        sheet_name="Duplicate_Review",
        index=False
    )

    invoice_mismatches.to_excel(
        writer,
        sheet_name="Invoice_Mismatches",
        index=False
    )
```

In [ ]:

```

```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF